Actividad 5 — Triage de defectos, análisis causa raíz, plan de mejora y quality gates

Sistema bajo prueba: Restful-Booker

Camilo Rivera Quintero

Karen Torres

Calidad en Software II · Universidad de La Salle · 30 de octubre de 2026

- 1 Introducción
- 2 Objetivo
- 3 Entradas: defectos y evidencias
  - 3.1 D-001 — Escritura sin autenticación real mediante header Basic hardcodeado
  - 3.2 D-002 — Se aceptan reservas con checkout anterior al checkin
  - 3.3 D-003 — totalprice negativo aceptado y no numérico convertido en null sin aviso
  - 3.4 D-004 — Se aceptan reservas solapadas (doble reserva)
  - 3.5 D-005 — DELETE sobre una reserva inexistente responde 405 en lugar de 404
- 4 Acta de triage
- 5 Análisis de causa raíz
  - 5.1 D-001 — Escritura sin autenticación real mediante header Basic hardcodeado
  - 5.2 D-002 — Se aceptan reservas con checkout anterior al checkin
  - 5.3 D-003 — totalprice negativo aceptado y no numérico convertido en null sin aviso
  - 5.4 D-004 — Se aceptan reservas solapadas (doble reserva)
  - 5.5 D-005 — DELETE sobre una reserva inexistente responde 405 en lugar de 404
- 6 Plan de mejora
- 7 Quality gates propuestos
- 8 Matriz de trazabilidad
- 9 Propuesta técnica de automatización en CI/CD
  - 9.1 Diseño del pipeline
  - 9.2 Herramientas seleccionadas y justificación
  - 9.3 Umbrales de seguridad del pipeline
  - 9.4 Configuración recomendada para el gate de SonarQube (G7)
- 10 Piloto automatizado: evidencia rojo → verde
- 11 Conclusiones
- 12 Referencias
- 13 Anexos

1 Introducción

Este informe documenta un ciclo completo de gestión de defectos sobre **Restful-Booker**, la API REST de reservas (Node.js/Express con LokiJS en memoria) que el equipo ha usado como sistema bajo prueba desde la Actividad 2. Allí se ejecutó el Master Test Plan y se confirmaron defectos reales, con evidencia curl y pruebas automatizadas. La Actividad 3 midió su rendimiento con k6 y la Actividad 4 consolidó esos KPI en un dashboard. Esta actividad toma esos hallazgos como **entrada** y los lleva por el flujo de calidad visto en clase:

```
Entradas (defectos + evidencia) → Triage → Causa raíz (5 porqués + Ishikawa) → Plan de mejora → Quality gates en CI/CD
```

Los defectos no son simulados: cada uno conserva el código con el que se registró en la Actividad 2 (DEF-0X, R-X) y recibe un código nuevo (**D-001 a D-005**) que se mantiene igual en el acta, el análisis causal, las acciones, los KPI, las pruebas de regresión y los gates del pipeline. Para asegurar esa trazabilidad, toda la información del triage vive en un único archivo de datos (`triage/defects.json`). Las tablas de este informe y el gate de triage del pipeline se generan a partir de él con la herramienta `triage/triage.py`.

Como valor agregado, además de proponer los quality gates se **implementó un piloto automatizado**: un pipeline de GitHub Actions (más un script equivalente para ejecución local) que se ejecutó primero en **rojo** sobre el código con los defectos y luego en **verde** después de corregir los cuatro defectos de mayor prioridad. Ambas ejecuciones quedaron registradas como evidencia.

2 Objetivo

Elaborar un acta de triage para cinco defectos reales de Restful-Booker, analizar sus causas raíz con los 5 porqués y el diagrama de Ishikawa, construir un plan de mejora priorizado por impacto/esfuerzo con responsables, esfuerzo y KPI de verificación, y proponer, implementar y evidenciar en un pipeline CI/CD los quality gates que previenen la recurrencia de esos defectos.

Alcance y supuestos

- Equipo: Camilo Rivera Quintero y Karen Torres. Los roles del comité de triage se describen en el acta.
- La aplicación es un fork académico de un proyecto de código abierto. Las correcciones se hicieron sobre una copia en el repositorio de esta actividad (`app/`), sin modificar el de actividades anteriores.
- El despliegue del pipeline es **simulado** (construcción de la imagen candidata). No existe un ambiente productivo real.

3 Entradas: defectos y evidencias

Los defectos provienen de la ejecución real del Master Test Plan de la Actividad 2 sobre Restful-Booker. Cada uno conserva su código de origen para mantener la trazabilidad entre actividades. Las referencias `archivo:línea` corresponden al código original, antes de las correcciones (commit `02c4ced`).

Código	Origen	Descripción	Impacto principal	Reproducibilidad	Alcance	Sev. propuesta	Prio. inicial
D-001	DEF-04 / Riesgo R3 (Actividad 2)	Escritura sin autenticación real mediante header Basic hardcodeado	Cualquier persona puede modificar o eliminar reservas ajenas: pérdida de integridad de datos, riesgo de seguridad y legal (datos personales de huéspedes).	100 % (determinista)	Todas las reservas y todos los clientes de la API	Crítica	P1
D-002	Riesgo R1 / caso T01 (Actividad 2)	Se aceptan reservas con checkout anterior al checkin	Datos de negocio inválidos: noches negativas, facturación y ocupación incorrectas, y reportes corruptos.	100 %	Todos los clientes que crean o actualizan reservas	Alta	P2
D-003	DEF-03 / Riesgo R4 (Actividad 2)	totalprice negativo aceptado y no numérico convertido en null sin aviso	Cobros incorrectos y corrupción silenciosa de datos financieros: la API responde 200 y el cliente cree que la reserva se guardó con su precio.	100 %	Todas las reservas creadas o actualizadas con payload malformado	Media	P2
D-004	DEF-01 / Riesgo R2 (Actividad 2)	Se aceptan reservas solapadas (doble reserva)	En un negocio real de reservas: doble venta de la misma habitación, sobrecupo, compensaciones económicas y pérdida de confianza.	100 %	Todas las reservas; el impacto real depende del modelo de habitaciones	Crítica	P1
D-005	DEF-02 / Riesgo R5 (Actividad 2)	DELETE sobre una reserva inexistente responde 405 en lugar de 404	Los clientes y la automatización interpretan mal el resultado (reintentos inútiles, alertas falsas); el contrato REST es inconsistente.	100 %	Clientes de integración que borran reservas	Media	P3

3.1 D-001 — Escritura sin autenticación real mediante header Basic hardcodeado

Módulo: `routes/index.js — PUT, PATCH y DELETE /booking/:id`

PUT, PATCH y DELETE /booking/:id aceptan el header 'Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=' (admin:password123 en Base64), publicado en la documentación de la propia API, sin pasar por POST /auth.

Pasos para reproducir: Levantar la API (docker compose up) → Sin llamar a /auth, enviar PUT /booking/1 con 'Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=' → Observar 200 OK y la reserva modificada

Resultado esperado: 403 Forbidden: solo se permite escribir con un token emitido por POST /auth.
Resultado obtenido: 200 OK (PUT/PATCH) y 201 (DELETE): la operación se ejecuta.

Tipo	Referencia	Qué muestra
curl	`triage/evidencia/T03_auth_hardcoded.txt`	PUT /booking/1 solo con el header Basic → 200 OK y la reserva se modifica
curl	`triage/evidencia/T-DELETE-01_auth_hardcoded.txt`	DELETE /booking/{id} solo con el header Basic → 201 y la reserva se elimina
código	`app/routes/index.js:434, :560, :615`	Condición que acepta el token de sesión o el literal 'Basic YWRt…', repetida en los tres handlers
prueba	`app/tests/spec.js:418-430 y :530-541`	Pruebas heredadas que esperan 200/201 usando el header Basic

3.2 D-002 — Se aceptan reservas con checkout anterior al checkin

Módulo: `helpers/validationrules.js, helpers/validator.js — POST y PUT /booking`

POST /booking crea reservas con fechas invertidas (checkout < checkin) o idénticas; la validación solo comprueba que los campos existan.

Pasos para reproducir: POST /booking con bookingdates.checkin = 2026-09-10 y checkout = 2026-09-01 → Observar 200 OK y la reserva creada

Resultado esperado: Rechazo (4xx) con el mensaje 'checkout debe ser posterior a checkin'.
Resultado obtenido: 200 OK y la reserva se persiste.

Tipo	Referencia	Qué muestra
curl	`triage/evidencia/T01_checkout_antes_checkin.txt`	checkin 2026-09-10, checkout 2026-09-01 → 200 OK, bookingid 11
código	`app/helpers/validationrules.js:7-8`	bookingdates.checkin/checkout solo con `presence: true`

3.3 D-003 — totalprice negativo aceptado y no numérico convertido en null sin aviso

Módulo: `helpers/validationrules.js, helpers/parser.js — POST y PUT /booking`

totalprice = -50 se guarda tal cual; totalprice = "abc" se acepta y parser.js lo convierte con parseInt en NaN, que se serializa como null.

Pasos para reproducir: POST /booking con totalprice = “abc” → Observar 200 OK y totalprice: null en la respuesta

Resultado esperado: Rechazo (4xx): totalprice debe ser numérico y ≥ 0.
Resultado obtenido: 200 OK; -50 se guarda y “abc” se convierte en null.

Tipo	Referencia	Qué muestra
curl	`triage/evidencia/T04_totalprice_negative.txt`	totalprice -50 → 200 OK, precio negativo persistido
curl	`triage/evidencia/T04_totalprice_string.txt`	totalprice “abc” → 200 OK con “totalprice”: null
código	`app/helpers/parser.js:24, :58`	‘totalprice’: parseInt(rawBooking.totalprice) sin comprobar NaN
código	`app/helpers/validationrules.js:5`	totalprice: {presence: true} sin numericality

3.4 D-004 — Se aceptan reservas solapadas (doble reserva)

Módulo: `routes/index.js POST /booking, models/booking.js`

Dos reservas para el mismo rango de fechas se crean sin conflicto. La API no tiene el concepto de habitación, por lo que no se puede decidir qué es un solapamiento.

Pasos para reproducir: POST /booking con checkin 2026-12-01 / checkout 2026-12-05 → Repetir el mismo POST → Observar dos reservas creadas

Resultado esperado: Rechazo (409 Conflict) si el recurso ya está reservado en ese rango.
Resultado obtenido: 200 OK en ambas solicitudes.

Tipo	Referencia	Qué muestra
prueba automatizada	`restful-booker/master-plan/automation (escenario Cucumber 'DEF-01') y master-plan/evidencias/cucumber_smoke_result.xml`	Dos POST con el mismo rango → dos bookingid distintos
código	`app/models/booking.js:24-33`	create() inserta sin consultar reservas existentes

3.5 D-005 — DELETE sobre una reserva inexistente responde 405 en lugar de 404

Módulo: `routes/index.js — DELETE /booking/:id`

Eliminar un id que no existe devuelve 405 Method Not Allowed, que significa ‘método no soportado’, en vez de 404 Not Found.

Pasos para reproducir: Obtener un token en POST /auth → DELETE /booking/10000000 con el token → Observar 405

Resultado esperado: 404 Not Found.
Resultado obtenido: 405 Method Not Allowed.

Tipo	Referencia	Qué muestra
curl	`triage/evidencia/T05_delete_nonexistent.txt`	DELETE /booking/999999 → 405 Method Not Allowed
código	`app/routes/index.js:622`	res.sendStatus(405) cuando no hay registro
prueba	`app/tests/spec.js:543–552`	Prueba heredada que espera 405

4 Acta de triage

Fecha: 2026-10-30 · **Modalidad:** Sesión de triage virtual (60 min)

Asistentes y roles:

- Camilo Rivera Quintero: Moderador del comité de triage / QA lead; responsable técnico de backend y CI
- Karen Torres: Analista de calidad y desarrolladora; registro del acta y validación funcional

Escala de severidad:

Severidad	Criterio
Crítica	Compromete seguridad, integridad de datos o una operación central del negocio sin alternativa.
Alta	Afecta de forma importante una funcionalidad o los datos del negocio; existe alguna mitigación parcial.
Media	Impacto funcional moderado; afecta la experiencia o la integración sin comprometer el negocio.
Baja	Defecto menor o cosmético.

Escala de prioridad:

Prioridad	Acción esperada
P1	Se atiende de inmediato (hotfix o ciclo actual, bloquea el despliegue).
P2	Se trabaja en el ciclo actual o en el siguiente release, no bloquea.
P3	Se programa para el próximo sprint.
P4	Backlog, se atiende según disponibilidad.

Criterios aplicados en la sesión:

- Severidad = gravedad del daño (seguridad, integridad de datos, operación del negocio). Prioridad = qué tan pronto actuar (urgencia, riesgo de release, existencia de workaround, esfuerzo y dependencias).
- Matriz impacto × urgencia (1–5): importante y urgente → P1; importante no urgente → P2; urgente no importante → P3; resto → P4.

- Reproducibilidad y alcance: un defecto 100 % reproducible que afecta a todos los usuarios de un endpoint sube en urgencia.
- Si existe un workaround operativo, la urgencia puede bajar, pero la severidad no cambia.
- Ningún defecto se cierra sin criterio de cierre verificable: prueba de regresión automatizada con el código del defecto en verde dentro del pipeline + smoke de rendimiento en verde.
- Análisis sin culpa (blameless, Google SRE): se buscan fallas de proceso y de sistema, no responsables individuales.

Decisiones acordadas:

Código	Sev. acordada	Prio. acordada	Impacto × urgencia (matriz)	Responsable	Decisión	Fecha compromiso	Workaround	Criterio de cierre
D-001	Crítica	P1	5×5 → P1	Camilo Rivera Quintero (backend)	Hotfix	2026-10-31	Ninguno técnico. Mitigación temporal: no exponer la API públicamente hasta desplegar el hotfix.	La prueba de regresión ‘D-001’ pasa (PUT/PATCH/DELETE con el header Basic → 403; con token válido → 200/201) y el escaneo de secretos (gitleaks) no reporta credenciales en el código.
D-002	Alta	P1	4×4 → P1	Karen Torres (validaciones)	Corrección en el ciclo actual	2026-11-03	Consulta diaria de reservas con fechas invertidas y corrección manual (paliativo, no corrige la causa).	La prueba de regresión ‘D-002’ pasa (fechas invertidas o iguales → rechazo; fechas válidas → 200) y el smoke k6 sigue en verde.
D-003	Alta	P1	4×4 → P1	Karen Torres (validaciones)	Corrección en el ciclo actual	2026-11-03	Ninguno confiable: el valor original se pierde al convertirse en null.	La prueba de regresión ‘D-003’ pasa (negativo y no numérico → rechazo; 0 y valores positivos → 200).

Código	Sev. acordada	Prio. acordada	Impacto × urgencia (matriz)	Responsable	Decisión	Fecha compromiso	Workaround	Criterio de cierre
D-004	Crítica	P2	5×2 → P2	Camilo Rivera Quintero (diseño) + Product Owner	Investigación y aclaración del requisito; corrección en el siguiente release	2026-11-14	Conciliación diaria de reservas con fechas superpuestas (consulta GET /booking?checkin=&checkout=) y contacto manual al cliente.	Requisito aprobado por el PO; prueba de regresión 'D-004' (misma habitación + rango solapado → 409; habitaciones distintas → 200) en verde.
D-005	Media	P3	2×2 → P4	Karen Torres	Quick win: se corrige junto con D-001 porque toca el mismo handler	2026-11-10	Los clientes pueden tratar el 405 de DELETE como 'no existe'.	La prueba de regresión 'D-005' pasa (DELETE de un id inexistente → 404).

Justificación de cada decisión:

- **D-001**: Vulnerabilidad de seguridad explotable con información pública y sin workaround. Importante y urgente: se hace primero.
- **D-002** *(propuesta: Alta/P2 → acordada: Alta/P1)*: Se propuso P2, pero el comité la sube a P1: corrompe datos de negocio en cada uso, es 100 % reproducible y su corrección es de esfuerzo bajo.
- **D-003** *(propuesta: Media/P2 → acordada: Alta/P1)*: Se reclasifica de Media a Alta: no es solo falta de validación, hay pérdida silenciosa de datos financieros (pérdida económica explícita). Sin workaround → P1.
- **D-004** *(propuesta: Crítica/P1 → acordada: Crítica/P2)*: La severidad sigue siendo Crítica, pero la prioridad baja a P2: corregirlo exige un cambio de modelo (entidad habitación/recurso) y una decisión de negocio. Un hotfix que rechace cualquier solapamiento global bloquearía reservas legítimas. Hay workaround operativo. Es el caso donde severidad y prioridad se separan.
- **D-005**: La matriz impacto × urgencia (2×2) sugiere P4 (backlog). El comité la sube a P3 porque rompe la automatización de los clientes de integración, y además la corrige como quick win dentro del hotfix de D-001: toca el mismo handler y el esfuerzo es mínimo (impacto bajo / esfuerzo bajo).

Hallazgos adicionales de la sesión:

- Tres pruebas existentes de tests/spec.js afirman el comportamiento defectuoso (aceptar el Basic hardcodeado en PUT y DELETE, y responder 405 al borrar un id inexistente). La suite 'pasaba en verde' precisamente porque protegía los defectos. Se decide reescribirlas junto con la corrección.
- POST /booking responde 500 (error de servidor) cuando la validación falla, en lugar de 400. Se registra como observación para backlog (P4), sin código propio en este ciclo, porque cambiarlo rompe el contrato con clientes existentes y requiere acordarlo.
- Hallazgo del propio pipeline: al implementar el gate G2, npm audit detectó 7 vulnerabilidades en dependencias de producción (1 crítica en proxy-addr, 2 altas en body-parser y path-to-regexp). Se corrigieron con actualizaciones minor/patch (npm audit fix; body-parser 2.4.0, morgan 1.12.1, uuid 13.0.2) y la suite siguió en verde. Queda una moderada en validate.js sin parche disponible: riesgo aceptado bajo el umbral 'high' y mitigado por la acción A-07 (reemplazar validate.js por un esquema JSON).

Seguimiento del estado:

Código	Estado	Fecha de cierre	Prueba de regresión
D-001	Resuelto	2026-10-30	D-001
D-002	Resuelto	2026-10-30	D-002
D-003	Resuelto	2026-10-30	D-003
D-004	Abierto	—	D-004
D-005	Resuelto	2026-10-30	D-005

5 Análisis de causa raíz

A cada defecto priorizado se le aplicaron los **5 porqués** (profundidad causal) y el **diagrama de Ishikawa** (amplitud de causas concurrentes por categoría). Cada causa se marca como **verificada** (con evidencia: código `archivo:línea`, log o prueba) o como **suposición por validar**. El análisis es *blameless*: las causas apuntan a procesos y sistemas, no a personas.

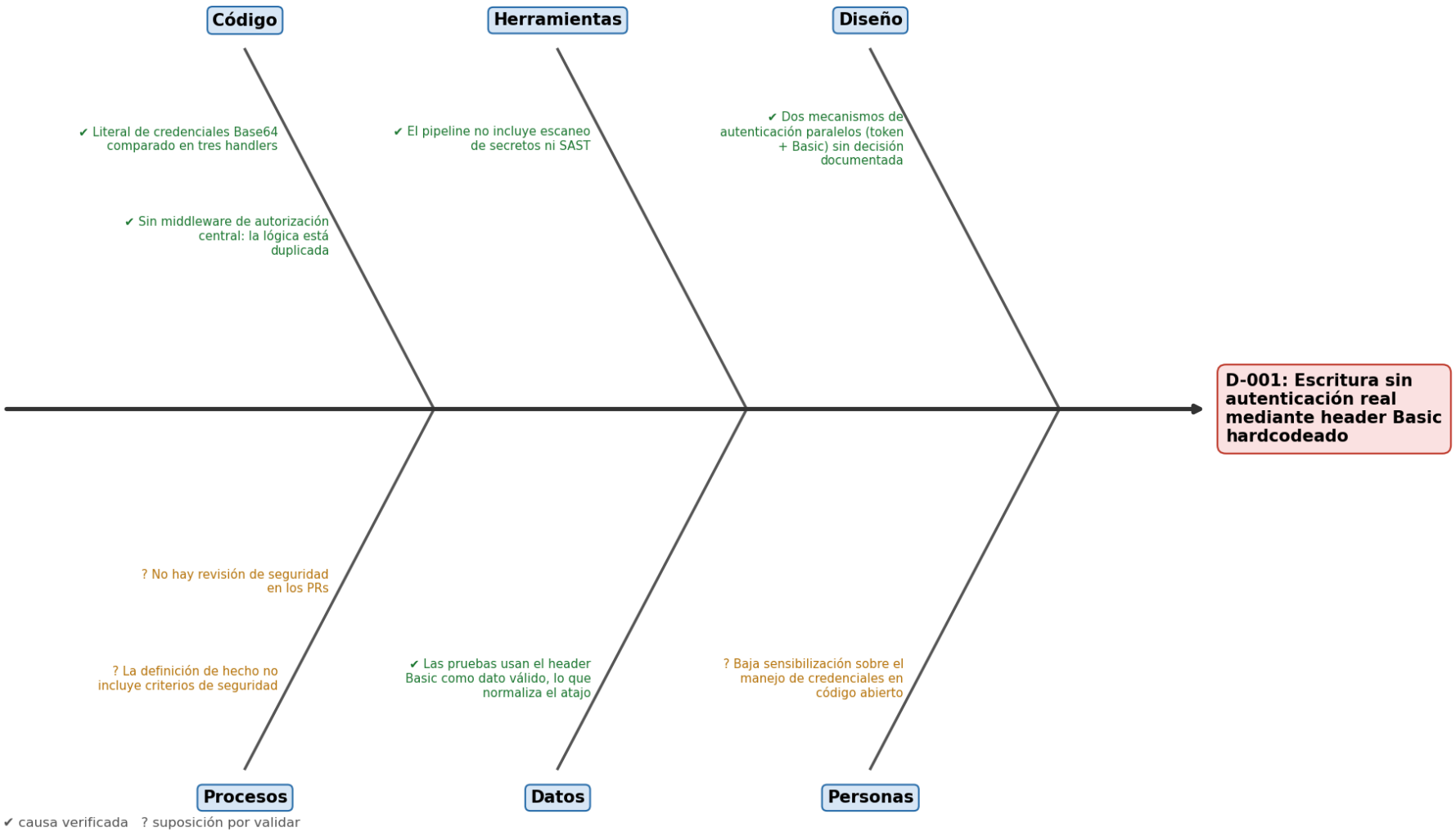
5.1 D-001 — Escritura sin autenticación real mediante header Basic hardcodeado

5 porqués

#	Pregunta	Respuesta
1	¿Por qué se puede modificar una reserva sin autenticarse?	Porque los handlers PUT/PATCH/DELETE aceptan como válido un header Basic con credenciales fijas, además del token de sesión.
2	¿Por qué los handlers aceptan un header con credenciales fijas?	Porque la condición de autorización compara el header contra un literal hardcodeado en el código (routes/index.js:434, :560, :615).
3	¿Por qué hay un literal de credenciales en el código?	Porque se añadió como atajo de autenticación para pruebas/demos y nunca se separó la configuración del código; tampoco existe un módulo de autorización central, así que la lógica se copió en tres handlers.
4	¿Por qué el atajo llegó al código que se despliega sin que nadie lo detectara?	Porque las pruebas existentes (tests/spec.js:418, :530) validan que el Basic funcione, por lo que el pipeline lo trata como comportamiento correcto, y no existe un escaneo de secretos ni una revisión de seguridad en CI.
5	¿Por qué no existe un control de seguridad en el pipeline?	Porque la definición de 'hecho' y los quality gates del proyecto solo exigían que las pruebas unitarias pasaran; no hay requisitos de seguridad explícitos ni gates que los verifiquen.

Causa raíz: Ausencia de requisitos y controles de seguridad en el proceso (definición de hecho y pipeline sin escaneo de secretos), sumada a una autorización duplicada y hardcodeada en cada handler.

Diagrama de Ishikawa



Ishikawa D-001

Categoría	Causa	Estado	Evidencia / cómo validar
Código	Literal de credenciales Base64 comparado en tres handlers	Verificada	app/routes/index.js:434, :560, :615
Código	Sin middleware de autorización central: la lógica está duplicada	Verificada	app/routes/index.js (misma condición en PUT/PATCH/DELETE)
Procesos	La definición de hecho no incluye criterios de seguridad	Suposición por validar	Por validar con el historial de PRs del proyecto original

Categoría	Causa	Estado	Evidencia / cómo validar
Procesos	No hay revisión de seguridad en los PRs	Suposición por validar	Por validar: no hay plantilla de PR ni CODEOWNERS en el repo
Herramientas	El pipeline no incluye escaneo de secretos ni SAST	Verificada	restful-booker/.github/workflows/test.yml solo ejecuta npm test
Datos	Las pruebas usan el header Basic como dato válido, lo que normaliza el atajo	Verificada	app/tests/spec.js:426, :538
Diseño	Dos mecanismos de autenticación paralelos (token + Basic) sin decisión documentada	Verificada	apidoc en routes/index.js documenta ambos (‘Basic auth’)
Personas	Baja sensibilización sobre el manejo de credenciales en código abierto	Suposición por validar	Por validar con una encuesta breve al equipo

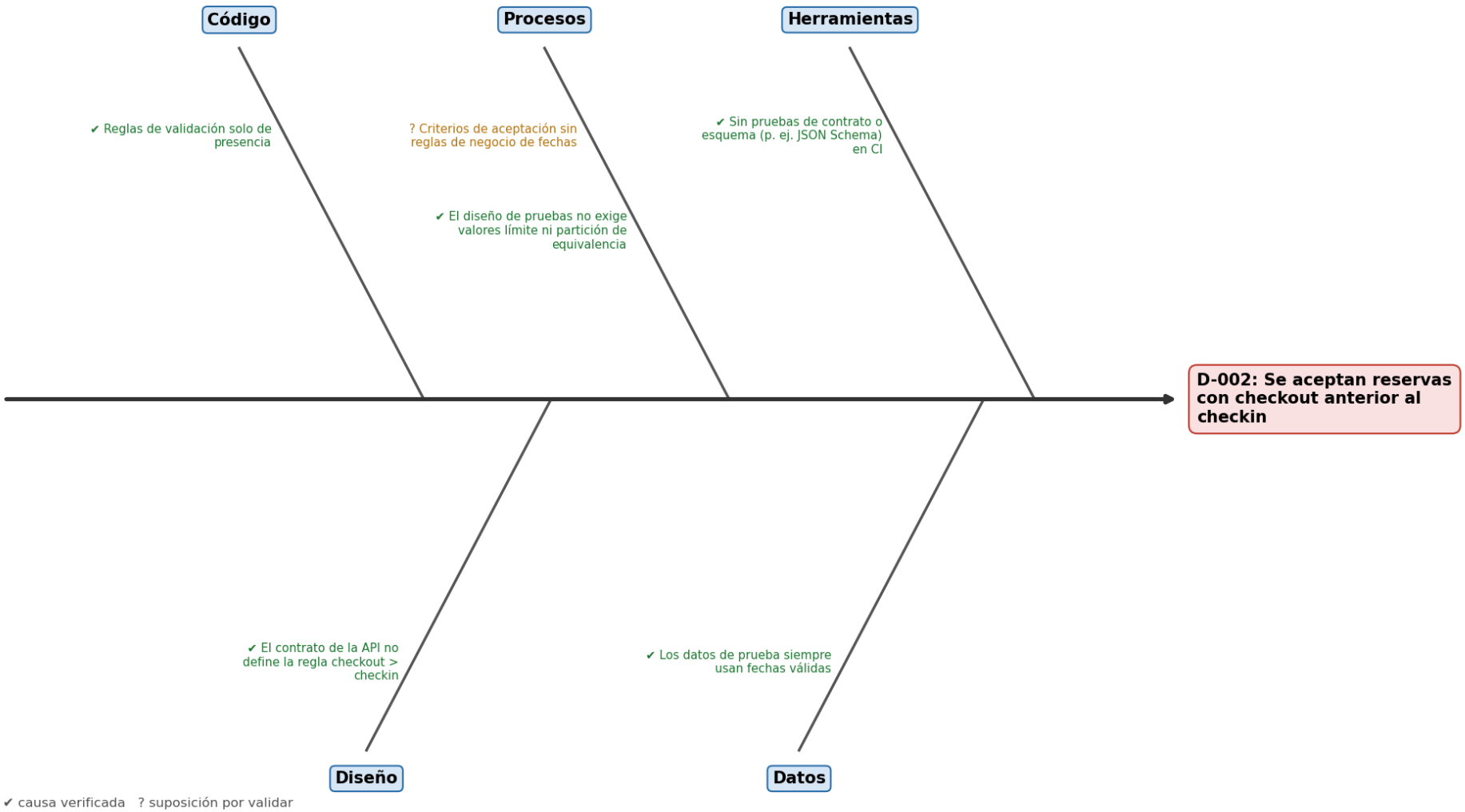
5.2 D-002 — Se aceptan reservas con checkout anterior al checkin

5 porqués

#	Pregunta	Respuesta
1	¿Por qué se guardó una reserva con checkout antes del checkin?	Porque el validador aceptó el payload como válido.
2	¿Por qué el validador lo aceptó?	Porque las reglas de validate.js solo exigen presencia de los campos de fecha (validationrules.js:7-8), no su orden ni su formato.
3	¿Por qué las reglas no cubren el orden de las fechas?	Porque las reglas de negocio de una reserva (checkout > checkin) no están especificadas como criterio de aceptación de la API; solo existe el contrato de campos.
4	¿Por qué no se detectó en pruebas?	Porque la suite solo prueba casos válidos con fechas correctas; no aplica partición de equivalencia ni valores límite sobre fechas.
5	¿Por qué no hay pruebas de valores límite?	Porque el proceso de diseño de pruebas no exige técnicas de caja negra para reglas de negocio ni pide cobertura de casos negativos como criterio de salida.

Causa raíz: Reglas de negocio de fechas no especificadas ni probadas: validación limitada a presencia de campos y ausencia de casos negativos/límite en el diseño de pruebas.

Diagrama de Ishikawa



Ishikawa D-002

Categoría	Causa	Estado	Evidencia / cómo validar
Código	Reglas de validación solo de presencia	Verificada	app/helpers/validationrules.js:7-8
Diseño	El contrato de la API no define la regla checkout > checkin	Verificada	apidoc POST /booking (routes/index.js) sin restricciones de fecha
Procesos	Criterios de aceptación sin reglas de negocio de fechas	Suposición por validar	Por validar con el backlog/historias del proyecto

Categoría	Causa	Estado	Evidencia / cómo validar
Procesos	El diseño de pruebas no exige valores límite ni partición de equivalencia	Verificada	app/tests/spec.js: ningún caso con fechas inválidas
Datos	Los datos de prueba siempre usan fechas válidas	Verificada	app/tests/spec.js:28-30 (payloads con checkin < checkout)
Herramientas	Sin pruebas de contrato o esquema (p. ej. JSON Schema) en CI	Verificada	restful-booker/.github/workflows/test.yml

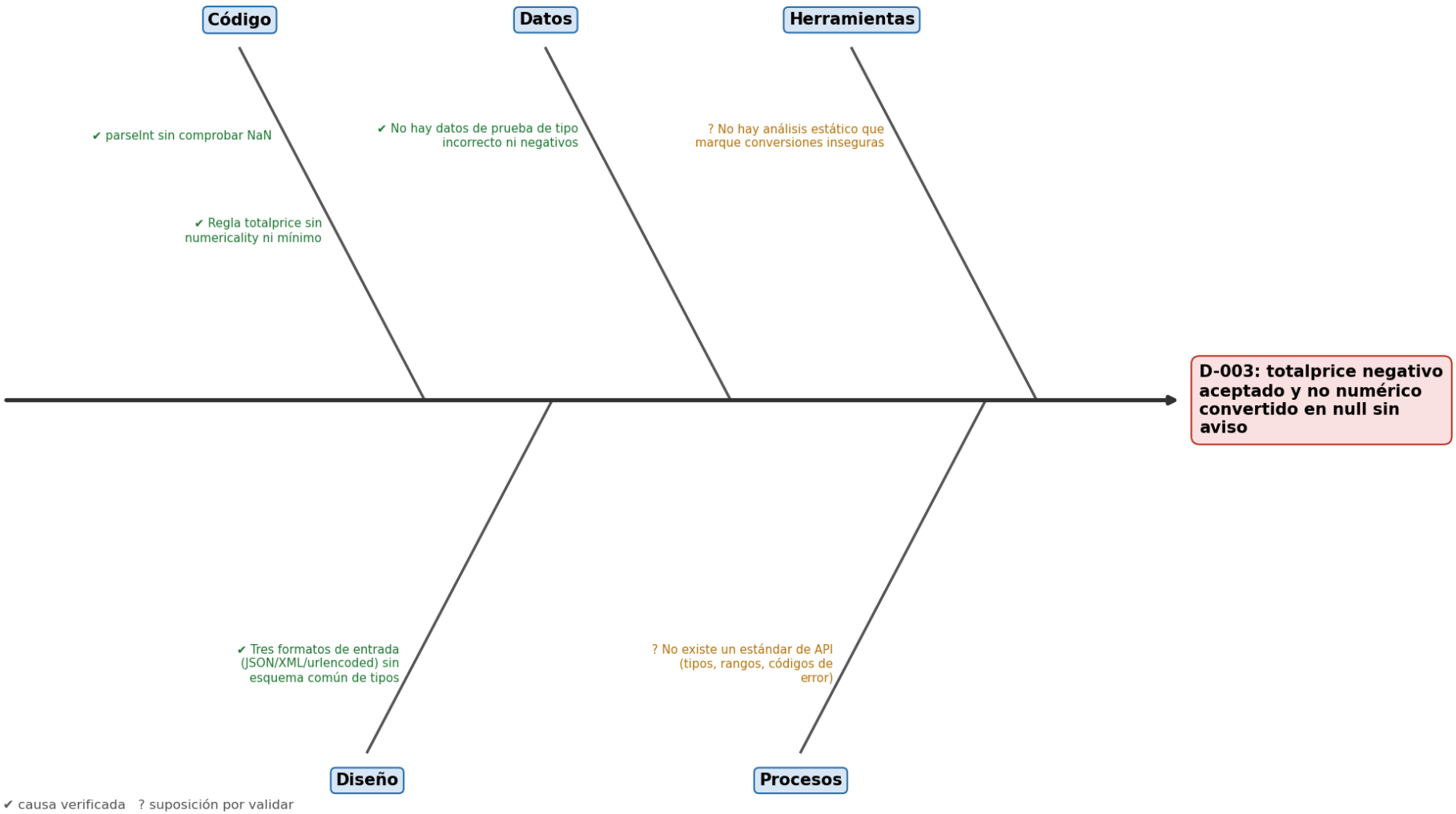
5.3 D-003 — totalprice negativo aceptado y no numérico convertido en null sin aviso

5 porqués

#	Pregunta	Respuesta
1	¿Por qué una reserva quedó con totalprice null?	Porque parser.js aplica parseInt a 'abc', que devuelve NaN, y JSON serializa NaN como null.
2	¿Por qué llegó un valor no numérico al parser?	Porque la regla de validación de totalprice solo exige presencia (validationrules.js:5), no tipo ni rango.
3	¿Por qué el parser no detecta el NaN?	Porque está diseñado para transformar la salida y no valida; asume que el dato ya viene limpio (responsabilidad repartida sin contrato).
4	¿Por qué no hay contrato de tipos entre la entrada y la persistencia?	Porque la API acepta JSON, XML y form-urlencoded (donde todo es texto) y no hay un esquema común que normalice y valide tipos antes de guardar.
5	¿Por qué no se definió ese esquema?	Porque no hay estándar de diseño de la API (tipos, rangos, códigos de error) ni un paso de revisión de contrato en el proceso de desarrollo.

Causa raíz: Falta de un contrato de datos (tipos y rangos) validado en la entrada; el parser convierte en silencio en lugar de fallar.

Diagrama de Ishikawa



Ishikawa D-003

Categoría	Causa	Estado	Evidencia / cómo validar
Código	parseInt sin comprobar NaN	Verificada	app/helpers/parser.js:24, :58
Código	Regla totalprice sin numericality ni mínimo	Verificada	app/helpers/validationrules.js:5
Diseño	Tres formatos de entrada (JSON/XML/urlencoded) sin esquema común de tipos	Verificada	app/app.js (parsers json, xml, urlencoded)

Categoría	Causa	Estado	Evidencia / cómo validar
Datos	No hay datos de prueba de tipo incorrecto ni negativos	Verificada	app/tests/spec.js: totalprice siempre 111
Procesos	No existe un estándar de API (tipos, rangos, códigos de error)	Suposición por validar	Por validar: no hay guía de estilo de API en el repo
Herramientas	No hay análisis estático que marque conversiones inseguras	Suposición por validar	Por validar con una regla de SonarQube/ESLint

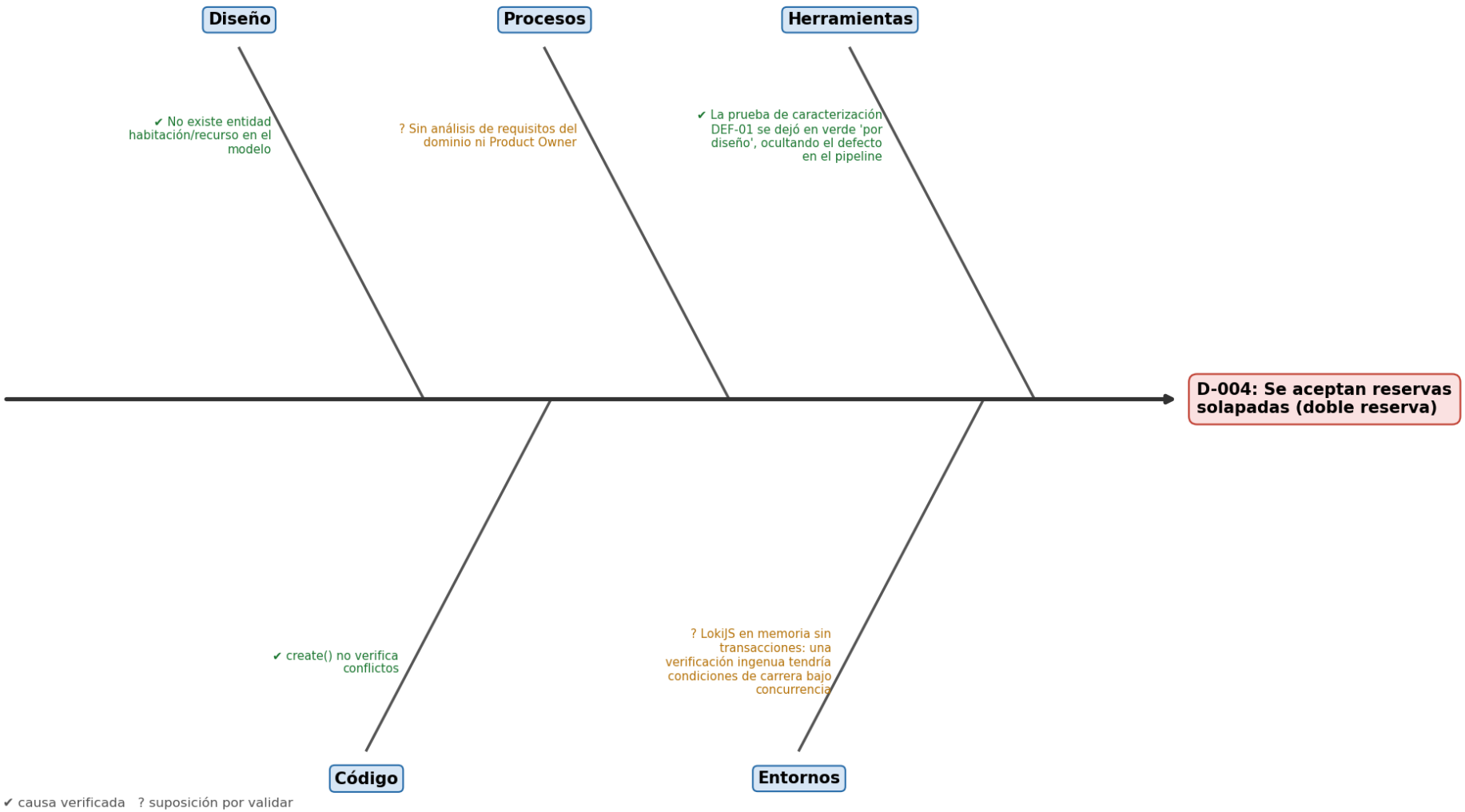
5.4 D-004 — Se aceptan reservas solapadas (doble reserva)

5 porqués

#	Pregunta	Respuesta
1	¿Por qué se aceptan dos reservas para las mismas fechas?	Porque create() inserta la reserva sin consultar si existe otra en el mismo rango.
2	¿Por qué no se consulta?	Porque el modelo de datos no tiene un recurso reservable (habitación), así que no hay contra qué comparar el solapamiento.
3	¿Por qué el modelo no tiene habitaciones?	Porque la API se diseñó como un sandbox de pruebas centrado en el CRUD de reservas, no en las reglas de disponibilidad del dominio.
4	¿Por qué no se identificó la brecha de dominio antes?	Porque no se hizo un análisis de requisitos del dominio de reservas; los requisitos se derivaron del contrato existente de la API.
5	¿Por qué los requisitos se derivaron del contrato?	Porque no hay un Product Owner ni un proceso de refinamiento que valide las reglas de negocio críticas (disponibilidad) antes de construir.

Causa raíz: Brecha de diseño del dominio: el modelo no representa el recurso reservable y no hubo análisis de requisitos de disponibilidad.

Diagrama de Ishikawa



Ishikawa D-004

Categoría	Causa	Estado	Evidencia / cómo validar
Diseño	No existe entidad habitación/recurso en el modelo	Verificada	app/models/booking.js (colección única ‘bookings’)
Código	create() no verifica conflictos	Verificada	app/models/booking.js:24-33
Procesos	Sin análisis de requisitos del dominio ni Product Owner	Suposición por validar	Por validar con el autor original / backlog

Categoría	Causa	Estado	Evidencia / cómo validar
Entornos	LokiJS en memoria sin transacciones: una verificación ingenua tendría condiciones de carrera bajo concurrencia	Suposición por validar	Por validar con la prueba de carga concurrente (k6, A3)
Herramientas	La prueba de caracterización DEF-01 se dejó en verde 'por diseño', ocultando el defecto en el pipeline	Verificada	restful-booker/master-plan/docs/MASTER_PLAN.md §9

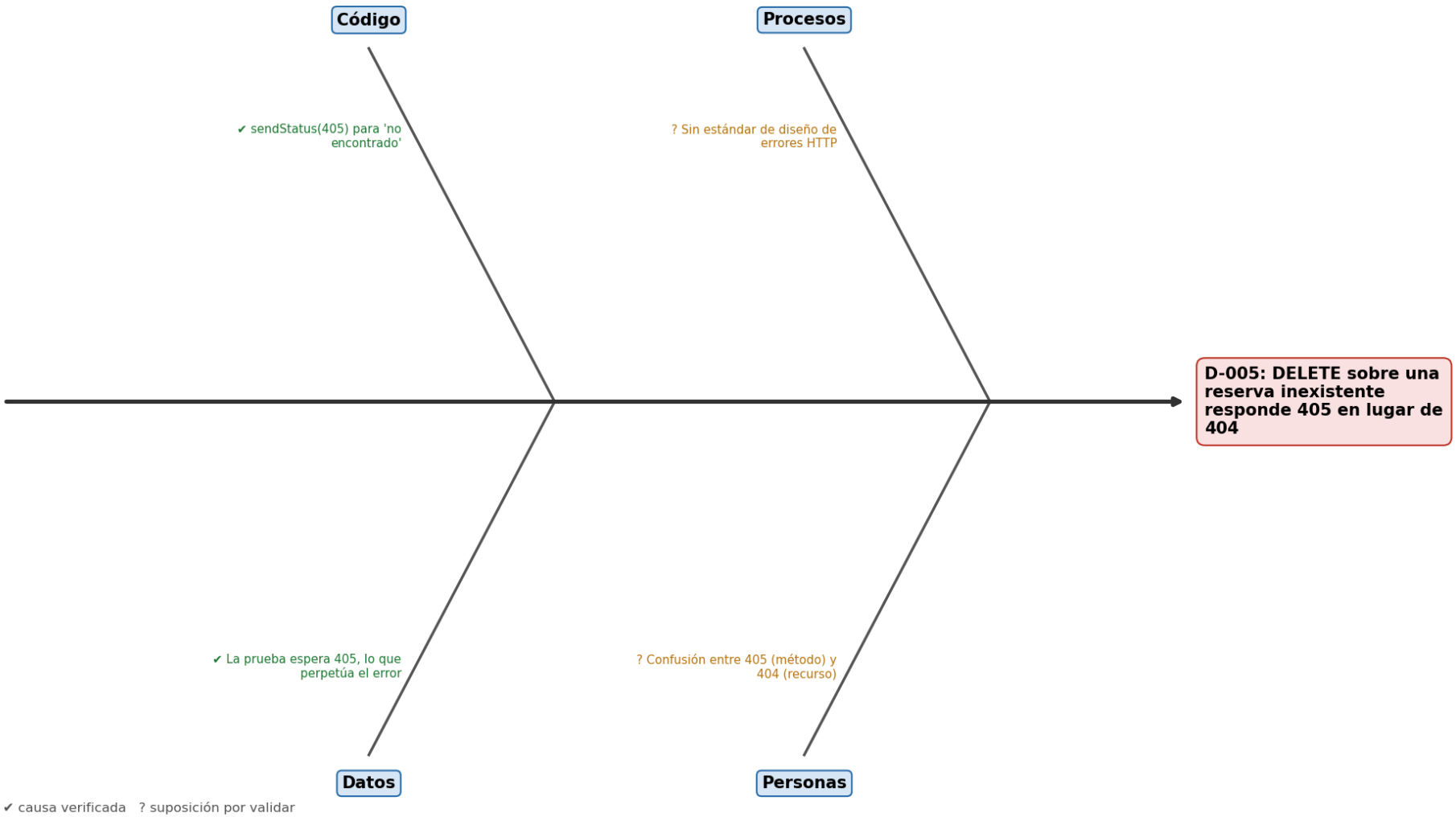
5.5 D-005 — DELETE sobre una reserva inexistente responde 405 en lugar de 404

5 porqués

#	Pregunta	Respuesta
1	¿Por qué DELETE de un id inexistente devuelve 405?	Porque el handler responde 405 cuando Booking.get no encuentra el registro (routes/index.js:622).
2	¿Por qué se eligió 405?	Porque se reutilizó el mismo código que en PUT/PATCH para 'registro no encontrado', sin consultar la semántica HTTP.
3	¿Por qué no se corrigió al revisarlo?	Porque la prueba heredada (spec.js:543) fija el 405 como resultado esperado, así que cualquier cambio rompía el build.
4	¿Por qué la prueba fija un código incorrecto?	Porque las pruebas se escribieron describiendo el comportamiento actual (caracterización), no a partir de una especificación de códigos de estado.
5	¿Por qué no hay especificación de códigos de estado?	Porque la API no tiene un estándar de diseño de errores; es la misma causa de proceso que en D-003.

Causa raíz: Ausencia de un estándar de códigos de estado HTTP; las pruebas de caracterización consolidaron el comportamiento incorrecto.

Diagrama de Ishikawa



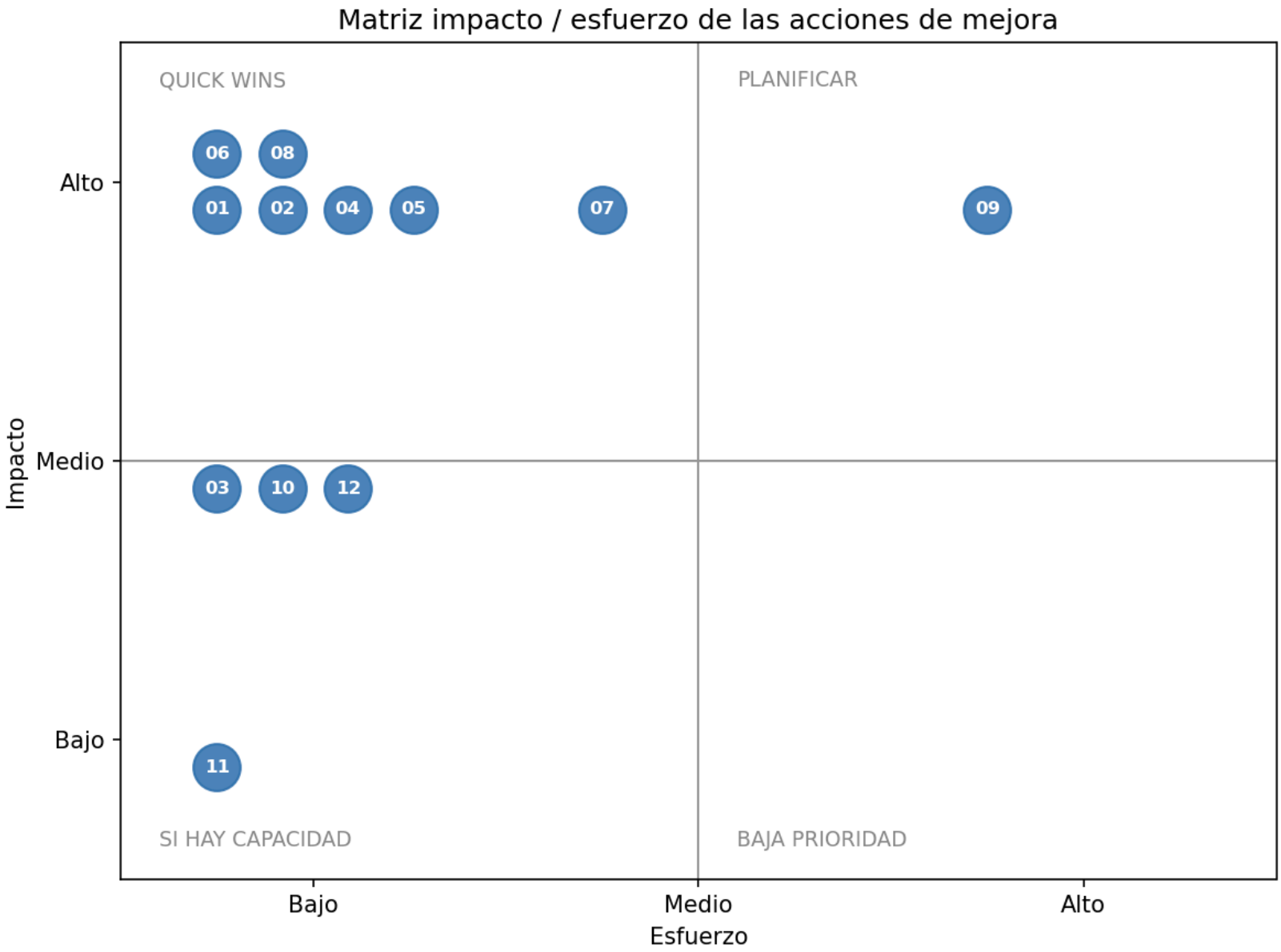
Ishikawa D-005

Categoría	Causa	Estado	Evidencia / cómo validar
Código	sendStatus(405) para 'no encontrado'	Verificada	app/routes/index.js:622
Datos	La prueba espera 405, lo que perpetúa el error	Verificada	app/tests/spec.js:551
Procesos	Sin estándar de diseño de errores HTTP	Suposición por validar	Por validar: no hay guía de API en el repo

Categoría	Causa	Estado	Evidencia / cómo validar
Personas	Confusión entre 405 (método) y 404 (recurso)	Suposición por validar	Por validar en la retrospectiva

6 Plan de mejora

Las acciones se ordenan con la matriz **impacto/esfuerzo**: primero los *quick wins* (alto impacto, bajo esfuerzo) y luego los proyectos planificados. Cada acción tiene un KPI con umbral y, cuando aplica, el quality gate que lo verifica de forma automática.



Matriz impacto/esfuerzo

Orden	Acción	Defecto	Tipo	Descripción	Responsable	Impacto	Esfuerzo	Plazo	KPI	Umbral	Gate	Cuadrante
1	A-01	D-001	Corrección	Eliminar la aceptación del header Basic hardcodeado; autorizar solo con el token de /auth mediante una única función isAuthorised (routes/index.js) y dejar de publicar la credencial en la apidoc	Camilo Rivera	Alto	Bajo	2026-10-31	Escrituras aceptadas sin token válido	0 (prueba D-001 en verde)	G3	Quick win (impacto alto/medio, esfuerzo bajo)
2	A-02	D-001	Prevención (herramienta)	Agregar escaneo de secretos con gitleaks como gate bloqueante del pipeline	Camilo Rivera	Alto	Bajo	2026-10-31	Secretos detectados en el código	0 hallazgos	G2	Quick win (impacto alto/medio, esfuerzo bajo)
3	A-04	D-002	Corrección	Validar que checkin y checkout sean fechas válidas y que checkout > checkin en validator.js (aplica a POST y PUT)	Karen Torres	Alto	Bajo	2026-11-03	Reservas con fechas inválidas aceptadas	0 (prueba D-002 en verde)	G3	Quick win (impacto alto/medio, esfuerzo bajo)
4	A-05	D-002	Pruebas adicionales	Agregar casos negativos y de valores límite (fechas iguales, invertidas, inválidas) a la suite de regresión	Karen Torres	Alto	Bajo	2026-11-03	Cobertura de líneas de helpers/ y routes/	≥ 80 %	G4	Quick win (impacto alto/medio, esfuerzo bajo)
5	A-06	D-003	Corrección	Regla numericality (≥ 0) para totalprice y rechazo de valores no numéricos antes de persistir	Karen Torres	Alto	Bajo	2026-11-03	Reservas con totalprice null o negativo	0 (prueba D-003 en verde)	G3	Quick win (impacto alto/medio, esfuerzo bajo)

Orden	Acción	Defecto	Tipo	Descripción	Responsable	Impacto	Esfuerzo	Plazo	KPI	Umbral	Gate	Cuadrante
6	A-08	D-004	Investigación	Taller con el PO para definir el recurso reservable y la regla de disponibilidad; documentar la decisión (ADR)	Camilo Rivera	Alto	Bajo	2026-11-07	Requisito de disponibilidad aprobado	ADR aprobado	—	Quick win (impacto alto/medio, esfuerzo bajo)
7	A-10	D-004	Workaround	Consulta diaria de reservas solapadas y conciliación manual mientras se corrige	Karen Torres	Medio	Bajo	2026-10-31	Solapamientos detectados y conciliados en < 24 h	100 %	—	Quick win (impacto alto/medio, esfuerzo bajo)
8	A-03	D-001	Prevención (proceso)	Añadir el checklist de seguridad (credenciales, autorización) a la definición de hecho y a la plantilla de PR	Karen Torres	Medio	Bajo	2026-11-07	% de PRs con checklist de seguridad completo	100 %	—	Quick win (impacto alto/medio, esfuerzo bajo)
9	A-12	D-005	Prevención (proceso)	Publicar una guía corta de códigos de estado (400/403/404/409) y revisarla en cada PR de la API	Karen Torres	Medio	Bajo	2026-11-10	Endpoints conformes a la guía	100 %	—	Quick win (impacto alto/medio, esfuerzo bajo)
10	A-07	D-003	Refactor	Definir un esquema JSON único de reserva (tipos, rangos) y validarlo para los tres formatos de entrada	Camilo Rivera	Alto	Medio	2026-11-14	Endpoints de escritura validados contra el esquema	100 %	—	Proyecto planificado (impacto alto/medio, esfuerzo medio/alto)
11	A-09	D-004	Refactor/infra	Agregar entidad habitación y verificación de solapamiento con índice por (habitación, fechas); responder 409	Camilo Rivera	Alto	Alto	2026-11-14	Reservas solapadas para la misma habitación	0 (prueba D-004 en verde)	G3	Proyecto planificado (impacto alto/medio, esfuerzo medio/alto)

Orden	Acción	Defecto	Tipo	Descripción	Responsable	Impacto	Esfuerzo	Plazo	KPI	Umbral	Gate	Cuadrante
12	A-11	D-005	Corrección	DELETE de un id inexistente → 404 y actualizar la prueba heredada	Karen Torres	Bajo	Bajo	2026-10-31	Respuestas 405 en DELETE de recursos inexistentes	0 (prueba D-005 en verde)	G3	Mejora rápida si hay capacidad

7 Quality gates propuestos

Gate	Etapa	Nombre	Qué mide	Umbral	Por qué este umbral	Previene	Herramienta	Si falla
G1	1. Estático (fail-fast)	Gate de triage	Defectos P1 abiertos; defectos resueltos sin prueba de regresión existente	0 P1 abiertos y 100 % de los resueltos con prueba ‘D-00X’	Traduce la regla de clase ‘no desplegar con defectos críticos abiertos’ a una regla automática conectada con el acta (defects.json).	D-001, D-002, D-003, D-004, D-005	triage/triage.py gate (Python stdlib)	El pipeline se detiene antes de compilar o probar.
G2	1. Estático (fail-fast)	Escaneo de secretos y dependencias	Credenciales en el código; vulnerabilidades altas/críticas en dependencias	0 secretos; 0 vulnerabilidades high/critical (npm audit)	D-001 nació de una credencial en el código. Es barato (segundos) y por eso va primero.	D-001	gitleaks + npm audit –audit-level=high	Bloquea el merge; el desarrollador debe retirar la credencial o actualizar la dependencia.
G3	2. Pruebas	Regresión de defectos + unitarias	Pruebas mocha (spec.js + regression.spec.js)	100 % de éxito (0 fallos)	Cada defecto cerrado tiene una prueba que impide que reaparezca (criterio de cierre del acta).	D-001, D-002, D-003, D-005	mocha + supertest	Bloquea; el reporte indica qué D-00X volvió a aparecer.
G4	2. Pruebas	Cobertura mínima	% de líneas cubiertas en helpers/, routes/ y models/	≥ 80 % de líneas	Umbral visto en clase y por defecto en el quality gate ‘Sonar way’ para código nuevo; las causas raíz de D-002/D-003 fueron ramas sin probar.	D-002, D-003	c8 (cobertura V8 nativa, sin instrumentar)	Bloquea hasta agregar las pruebas faltantes.

Gate	Etapa	Nombre	Qué mide	Umbral	Por qué este umbral	Previene	Herramienta	Si falla
G5	3. Rendimiento	Smoke de rendimiento	Tasa de error, latencia P95 y checks funcionales bajo 10 VU durante 30 s	http_req_failed < 1 %, p(95) < 300 ms, checks > 99 %	En la A3 se midió un P95 de 6–10 ms con un SLO de 800 ms. 300 ms es el umbral visto en clase y deja margen para detectar regresiones grandes (p. ej. validaciones costosas o la futura verificación de solapamiento de D-004) sin falsos positivos.	D-002, D-003, D-004	k6 (thresholds → exit code ≠ 0)	Bloquea el despliegue; se adjunta el resumen k6 como artefacto.
G6	4. Despliegue	Despliegue condicionado	Que G1–G5 estén aprobados	Todos en verde	Implementa el flujo de clase: pruebas → quality gates → ¿cumple? → despliegue / se detiene.	—	GitHub Actions (needs:) + docker build	No se construye ni publica la imagen.
G7	Recomendado (no implementado)	Quality gate SonarQube/SonarCloud	Cobertura de código nuevo, duplicación, issues bloqueantes y hotspots de seguridad	Cobertura nuevo código ≥ 80 %, duplicación < 3 %, 0 issues blocker/critical, rating de seguridad A	Es el estándar de la industria para gates estáticos y cubre la causa de D-001 (hotspot de credenciales) y D-003 (conversiones inseguras). No se implementa en el piloto porque requiere cuenta y token del servidor.	D-001, D-003	SonarCloud + sonarsource/sonarqube-scan-action	El estado del quality gate en el PR queda ‘Failed’ y bloquea el merge (branch protection).

8 Matriz de trazabilidad

Defecto	Evidencia	Sev./Prio.	Causa raíz	Acciones	Prueba de regresión	Gates	Estado
D-001	triage/evidencia/T03_auth_hardcoded.txt	Crítica/P1	Ausencia de requisitos y controles de seguridad en el proceso (definición de hecho y pipeline sin escaneo de secretos), sumada a una autorización duplicada y hardcodeada en cada handler.	A-01, A-02, A-03	D-001	G1, G2, G3, G7	Resuelto
D-002	triage/evidencia/T01_checkout_antes_checkin.txt	Alta/P1	Reglas de negocio de fechas no especificadas ni probadas: validación limitada a presencia de campos y ausencia de casos negativos/límite en el diseño de pruebas.	A-04, A-05	D-002	G1, G3, G4, G5	Resuelto
D-003	triage/evidencia/T04_totalprice_negative.txt	Alta/P1	Falta de un contrato de datos (tipos y rangos) validado en la entrada; el parser convierte en silencio en lugar de fallar.	A-06, A-07	D-003	G1, G3, G4, G5, G7	Resuelto
D-004	restful-booker/master-plan/automation (escenario Cucumber ‘DEF-01’) y master-plan/evidencias/cucumber_smoke_result.xml	Crítica/P2	Brecha de diseño del dominio: el modelo no representa el recurso reservable y no hubo análisis de requisitos de disponibilidad.	A-08, A-09, A-10	D-004	G1, G5	Abierto
D-005	triage/evidencia/T05_delete_nonexistent.txt	Media/P3	Ausencia de un estándar de códigos de estado HTTP; las pruebas de caracterización consolidaron el comportamiento incorrecto.	A-11, A-12	D-005	G1, G3	Resuelto

9 Propuesta técnica de automatización en CI/CD

9.1 Diseño del pipeline

Los quality gates convierten los criterios de cierre del acta y los KPI del plan de mejora en reglas automáticas. El pipeline aplica una política **fail-fast**: las etapas van de la más barata a la más costosa y cada una depende de la anterior (`needs:` en GitHub Actions). Si un gate falla, el resto del flujo y el despliegue no se ejecutan.

```
push / pull request
   │
   ├─ Etapa 1 · Estático (≈ 20 s) ── G1 gate de triage · G2 gitleaks + npm audit
   │        ✗ → se detiene
   ├─ Etapa 2 · Pruebas (≈ 30 s) ─── G3 regresión D-00X + unitarias · G4 cobertura ≥ 80 %
   │        ✗ → se detiene
   ├─ Etapa 3 · Rendimiento (≈ 1 min) — G5 docker build + k6 smoke (error < 1 %, p95 < 300 ms)
   │        ✗ → se detiene
   └─ Etapa 4 · Despliegue ──────────── G6 solo en main y con G1–G5 aprobados
```

- **Workflow:** `.github/workflows/quality-gate.yml` (se ejecuta en cada push y pull request a `main`).
- **Ejecución local equivalente:** `bash ci/run_gates.sh` ejecuta los mismos gates en el mismo orden. Con `FAIL_FAST=0` funciona en modo diagnóstico: ejecuta todos los gates y reporta cada resultado.
- **Artefactos:** reporte de cobertura (lcov) y resumen de k6 (`k6-summary.json`) publicados en cada ejecución.

9.2 Herramientas seleccionadas y justificación

Herramienta	Gate	Por qué se eligió	Alternativas consideradas
GitHub Actions	Orquestación (G1–G6)	El repositorio ya vive en GitHub y las actividades 2–4 usaron Actions. Es gratis para repositorios públicos y `needs:` implementa fail-fast de forma nativa.	GitLab CI, Jenkins (requiere servidor propio)
`triage.py` (Python stdlib)	G1	Conecta el acta con el pipeline: lee `defects.json` y bloquea si hay defectos P1 abiertos o defectos cerrados sin prueba de regresión. No tiene dependencias.	Integración con la API de Jira (requiere credenciales)
gitleaks	G2	Escáner de secretos open source y rápido (≈ 40 ms sobre la app). Permite reglas propias: se añadió `hardcoded-basic-auth`, dirigida a la causa raíz de D-001.	TruffleHog, GitHub secret scanning (no admite reglas propias en el plan gratuito)

Herramienta	Gate	Por qué se eligió	Alternativas consideradas
npm audit	G2	Viene incluido con npm y no requiere configuración. Bloquea vulnerabilidades high/critical en dependencias de producción.	Dependabot (avisa pero no bloquea), Snyk (requiere cuenta)
mocha + supertest	G3	Es el framework que ya usa el proyecto. Cada prueba de regresión lleva el código del defecto en su nombre, lo que da trazabilidad directa con el acta.	Jest (implicaría migrar la suite)
c8	G4	Usa la cobertura nativa de V8, sin instrumentar el código, y con `--check-coverage --lines 80` falla el build por debajo del umbral.	nyc/istanbul
k6	G5	Es la herramienta de la Actividad 3, así que los umbrales salen de una línea base medida. Sus thresholds devuelven un exit code ≠ 0, lo que implementa el fail-fast de forma natural.	JMeter (más pesado para CI), Artillery
SonarQube / SonarCloud	G7 (recomendado)	Es el estándar para gates estáticos: cobertura de código nuevo, duplicación < 3 %, issues bloqueantes y hotspots de seguridad. Se recomienda como siguiente paso. No se implementó porque requiere cuenta, token y configurar el servidor.	—

9.3 Umbrales de seguridad del pipeline

Indicador	Umbral	Origen del umbral
Defectos P1 abiertos	0	Regla de clase: no desplegar con defectos críticos o urgentes abiertos
Defectos cerrados sin prueba de regresión	0	Criterio de cierre del acta
Secretos en el código	0	Causa raíz de D-001
Vulnerabilidades high/critical en dependencias	0	Buenas prácticas OWASP. Las moderadas sin parche se registran como riesgo aceptado.
Pruebas fallidas	0 (100 % de éxito)	Criterio de cierre de D-001, D-002, D-003 y D-005
Cobertura de líneas (helpers, routes, models)	≥ 80 %	Umbral visto en clase y por defecto en “Sonar way” para código nuevo. La línea base medida fue 91 %.
Tasa de error HTTP (k6)	< 1 %	SLO de la Actividad 3
Latencia P95 (k6)	< 300 ms	Umbral de clase. La Actividad 3 midió 6–10 ms con un SLO de 800 ms, así que el margen detecta regresiones grandes sin falsos positivos.

Indicador	Umbral	Origen del umbral
Checks funcionales del smoke	> 99 %	Los flujos críticos (crear, consultar, actualizar con token) deben funcionar bajo carga

9.4 Configuración recomendada para el gate de SonarQube (G7)

```yaml
# Paso adicional sugerido para el job "estatico"
- uses: SonarSource/sonarqube-scan-action@v5
  env:
    SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
  with:
    args: >
      -Dsonar.projectKey=restful-booker-a5
      -Dsonar.sources=app/routes,app/helpers,app/models
      -Dsonar.javascript.lcov.reportPaths=app/coverage/lcov.info
      -Dsonar.qualitygate.wait=true
```

Condiciones del quality gate en el servidor: cobertura de código nuevo ≥ 80 %, líneas duplicadas en código nuevo < 3 %, 0 issues blocker/critical y rating de seguridad A. Con `sonar.qualitygate.wait=true`, el job falla si el gate no se aprueba.

10 Piloto automatizado: evidencia rojo → verde

El piloto se ejecutó sobre el historial de commits del repositorio de la actividad:

Commit	Contenido	Resultado del pipeline
`02c4ced`	App original + pruebas de regresión D-00X + gates	**ROJO** (`triage/evidencia/pipeline_01_rojo.log`)
`2389430`	Corrección de D-001, D-002, D-003 y D-005 + actualización de dependencias	**VERDE** (`triage/evidencia/pipeline_02_verde.log`)
`aca97e1`	Workflow de GitHub Actions con los mismos gates	—

Resultado por gate

Gate	Ejecución en rojo (código original)	Ejecución en verde (después de las correcciones)
G1 triage	✗ BLOQUEADO: D-001, D-002 y D-003 son P1 y están abiertos	✓ 0 P1 abiertos. Los 4 defectos resueltos tienen prueba de regresión. D-004 (P2) sigue abierto sin bloquear.

Gate	Ejecución en rojo (código original)	Ejecución en verde (después de las correcciones)
G2 secretos	✗ 19 hallazgos de la credencial Basic en el código y en la apidoc	✓ 0 hallazgos. npm audit: 0 high/critical (quedó 1 moderada sin parche, como riesgo aceptado)
G3 pruebas	✗ 36 pasan / **10 fallan** (todas las pruebas D-00X de defectos abiertos)	✓ 46 pasan / 0 fallan / 1 pendiente (D-004)
G4 cobertura	91,2 % de líneas	✓ 89,8 % de líneas (≥ 80 %)
G5 k6 smoke	✓ p95 = 7,51 ms; error 0,00 %	✓ p95 = 8,48 ms; error 0,00 %; checks 100 %
G6 despliegue	✗ Bloqueado	✓ Habilitado

Fragmento del log en rojo (G1):

```
D-001    Crítica  P1     open       no
D-002    Alta     P1     open       no
D-003    Alta     P1     open       no
D-004    Crítica  P2     open       no
D-005    Media    P3     open       no
RESULTADO: BLOQUEADO
  ✗ D-001 es P1 y está 'open' (debe estar resuelto antes de desplegar)
```

Verificación de cierre con los casos originales. Se repitieron los curl de la evidencia de la Actividad 2 contra la versión corregida (`triage/evidencia/verificacion_post_correccion.txt`). El PUT y el DELETE con el header Basic ahora responden **403**, como esperaba el criterio de cierre (antes 200/201). Las fechas invertidas y `totalprice` –50 o `"abc"` ahora se **rechazan** (antes 200 con datos corruptos). El DELETE de un id inexistente responde **404** (antes 405). Los flujos válidos con token siguen respondiendo 200.

Lecciones del piloto

1. Tres pruebas heredadas **afirmaban el comportamiento defectuoso**: aceptar el Basic y responder 405. La suite estaba en verde precisamente porque protegía los defectos. El gate G1 cierra esa brecha, porque un defecto solo puede cerrarse si existe una prueba con su código.
2. Las pruebas de regresión necesitan **aserciones precisas**. Al principio, dos casos de D-002 pasaban por la razón equivocada: sin header `Accept`, la API respondía 418 antes de validar. Se corrigió fijando `Accept: application/json` y verificando además que la reserva no se persistiera.
3. El gate G2 encontró **un problema que no estaba en el acta**: 7 vulnerabilidades en dependencias (1 crítica, 2 altas). Esto muestra el valor preventivo de los gates más allá de los defectos conocidos.
4. **Severidad ≠ prioridad.** D-004 es Crítico pero P2: el pipeline lo deja pasar de forma explícita y justificada, mientras el plan de mejora define su investigación y su workaround.

Cómo reproducir

```
cd app && npm ci && cd ..
FAIL_FAST=0 bash ci/run_gates.sh            # diagnóstico de todos los gates
python3 triage/triage.py gate               # solo el gate de triage
git checkout 02c4ced && bash ci/run_gates.sh  # reproducir el estado en rojo
```

11 Conclusiones

- El triage separó con criterios explícitos la gravedad (severidad) de la urgencia (prioridad). Dos defectos se reclasificaron durante la sesión (D-002 subió a P1 y D-003 pasó a Alta por pérdida silenciosa de datos financieros), y D-004 quedó como Crítico pero P2 porque su corrección requiere una decisión de diseño.
- El análisis causal mostró que las causas raíz son de **proceso** más que de código: no hay requisitos de seguridad ni de reglas de negocio, no hay estándar de API y la suite caracterizaba los defectos en lugar de especificar el comportamiento esperado. Por eso el plan combina correcciones con acciones preventivas de proceso y de herramientas.
- Los quality gates cierran el ciclo de mejora continua: cada criterio de cierre y cada KPI del plan tiene una regla automática que impide que el defecto vuelva a llegar a producción. El piloto demostró el ciclo completo, del rojo al verde, con evidencia reproducible.

12 Referencias

- ISTQB. (2024). *Certified Tester Advanced Level — Test Management (CTAL-TM) Syllabus v3.0*, sección Defect Management. International Software Testing Qualifications Board. https://www.istqb.org/
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Postmortem culture: Learning from failure. En *Site Reliability Engineering*. O'Reilly / Google. https://sre.google/sre-book/postmortem-culture/
- American Society for Quality. (s. f.). *Five Whys and Five Hows*. https://asq.org/quality-resources/five-whys
- NHS Improvement Cymru. (s. f.). *Cause and effect (fishbone) diagram*. Guía de herramientas de mejora de la calidad.
- SonarSource. (s. f.). *Quality gates*. Documentación de SonarQube. https://docs.sonarsource.com/
- Grafana Labs. (s. f.). *Thresholds*. Documentación de k6. https://grafana.com/docs/k6/latest/using-k6/thresholds/
- Gitleaks. (s. f.). *Gitleaks*. https://github.com/gitleaks/gitleaks

13 Anexos

- **A. Fuente de datos del triage:** `triage/defects.json`
- **B. Herramienta de triage:** `triage/triage.py` (`validate`, `gate`, `report`, `diagrams`)
- **C. Evidencia original (Actividad 2):** `triage/evidencia/T01…T05*.txt`
- **D. Logs del pipeline:** `triage/evidencia/pipeline_01_rojo.log`, `triage/evidencia/pipeline_02_verde.log`

- **E. Verificación posterior a la corrección:** `triage/evidencia/verificacion_post_correccion.txt`
- **F. Pruebas de regresión:** `app/tests/regression.spec.js`
- **G. Pipeline:** `.github/workflows/quality-gate.yml`, `ci/run_gates.sh`, `ci/k6_smoke.js`, `.gitleaks.toml`